

Non-vanishing support for the half-horizontal-domino constrained tiling count sequence on the $4 \times n$ board

(Lean 4 machine-checked claim of record)

Aurora0134*

September 30, 2026

Abstract

We record a machine-checked result about a pure finite-state integer recursion: the sequence $a_3(n) = D_4(n, 0, n)$ built from a column-mask transfer table has non-vanishing support exactly at the even indices, that is $a_3(n) \neq 0 \iff n \bmod 2 = 0$, together with two state-level parity lemmas that every reachable state has an even mask popcount and an even accumulated horizontal-tile count. All statements are verified by the Lean 4 kernel under the pinned mathlib revision, with no `sorry` and axioms confined to `{propext, Quot.sound, Classical.choice}`. Novelty: search found no occupying record. This note is a priority claim of record, not a full paper.

1 Introduction

This note records a formalized result produced by an AI4Math automated pipeline, in which propositions and proofs are generated by a language model and adjudicated solely by the Lean 4 kernel. The result concerns a finite-state integer recursion: a transfer table on column masks generates a two-parameter state count, from which the one-parameter sequence a_3 is read off; the theorems are a parity invariant for reachable states and the exact non-vanishing support of a_3 . The note is a priority claim of record, deposited for timestamping; the corresponding narrative paper has been deliberately

*Contact: via the GitHub account Aurora0134; ORCID placeholder — to be filled in by the author before any upload. No personal information is carried in this note.

deferred by the author (see the card), so no full exposition is attempted here. Verification strength is stated in Section 4: strict kernel compilation, zero **sorry**, standard axiom whitelist. The definitions never mention tilings, dominoes or counting, so the kernel layer is a pure integer recursion. No literature survey is included—that expectation belongs to arXiv moderation and does not apply to a Zenodo deposit.

2 Statement

We first fix notation. The recursion is built from a finite transfer table **steps4** of 29 triples $(occ, next, j)$, where occ is the incoming column mask, $next$ the outgoing column mask and j the number of horizontal tiles protruding from that column; the table is generated by exhaustive enumeration of the four-row column-filling recursion rather than transcribed by hand. Writing pc for the popcount of a mask in the range 0..15, and **0** for the empty mask, define the state count

$$D_4(0, mask, h) = \begin{cases} 1, & mask = 0 \wedge h = 0, \\ 0, & \text{otherwise,} \end{cases} \quad D_4(n+1, mask, h) = \sum_{\substack{(occ, next, j) \in \mathbf{steps4} \\ next = mask, j \leq h}} D_4(n, occ, h-j),$$

and the one-parameter sequence

$$a_3(n) = D_4(n, \mathbf{0}, n).$$

The guard $j \leq h$ is not removable: subtraction on $\mathbb{N}$ truncates, which would pollute the count. The following five facts are formalized. The first two are state-level parity lemmas, and the last is the main result.

Theorem 1 (reachable masks have even popcount; support lemma). *For all $n, mask, h \in \mathbb{N}$, if $D_4(n, mask, h) \neq 0$ then $2 \mid pc(mask)$.*

Theorem 2 (reachable states have even accumulated count; support lemma). *For all $n, mask, h \in \mathbb{N}$, if $D_4(n, mask, h) \neq 0$ then $2 \mid h$.*

Theorem 3 (vanishing at odd indices). *For every $n \in \mathbb{N}$, if $n \bmod 2 = 1$ then $a_3(n) = 0$.*

Theorem 4 (non-vanishing at even indices). *For every $k \in \mathbb{N}$, $a_3(2k) \neq 0$.*

Theorem 5 (support characterization; main result). *For every $n \in \mathbb{N}$, $a_3(n) \neq 0 \iff n \bmod 2 = 0$.*

Listing 1: Frozen formal statement (Lean 4, verbatim from `formalized/03-half4-statements.lean`, lines 30–103), with the five proof-body placeholder lines omitted (see the note below the listing).
sha256 b142670a...).

```

1  import Mathlib
2
3  /-- ** $m=4$  的列填充转移表**: `(occ, next, j)` 表示「入掩码  $\rightarrow$  出掩码, 该列伸出的水平砖数」。  
4  穷举自 4 行的列填充递归（`phase0/audit-lean-steps.py` 生成, 非手抄）。 -/
5  def steps4 : List (ℕ × ℕ × ℕ) :=
6    [(0, 0, 0), (0, 12, 2), (0, 9, 2), (0, 3, 2), (0, 15, 4),
7     (1, 8, 1), (1, 2, 1), (1, 14, 3),
8     (2, 1, 1), (2, 13, 3),
9     (3, 0, 0), (3, 12, 2),
10    (4, 8, 1), (4, 11, 3),
11    (5, 10, 2),
12    (6, 9, 2),
13    (7, 8, 1),
14    (8, 4, 1), (8, 1, 1), (8, 7, 3),
15    (9, 0, 0), (9, 6, 2),
16    (10, 5, 2),
17    (11, 4, 1),
18    (12, 0, 0), (12, 3, 2),
19    (13, 2, 1),
20    (14, 1, 1),
21    (15, 0, 0)]
22
23  /-- **掩码位数 `pc` (0..15 上的 popcount) **: 显式列举（避免依赖额外位运算 API）。 -/
24  def pc : ℕ → ℕ
25    | 0 => 0
26    | 1 => 1
27    | 2 => 1
28    | 3 => 2
29    | 4 => 1
30    | 5 => 2
31    | 6 => 2
32    | 7 => 3
33    | 8 => 1
34    | 9 => 2
35    | 10 => 2
36    | 11 => 3
37    | 12 => 2

```

```

38 | 13 => 3
39 | 14 => 3
40 | 15 => 4
41 | _ => 0
42
43 /-- **T3 的有限状态整数递推 `D4` **:
44 `D4 n mask h` = 填满前  $n$  列、出掩码 =  $mask$ 、累计水平砖数 =  $h$  的走法
    数。
45
46 注意 `j ≤ h` 保护不可省:  $\mathbb{N}$  减法截断会污染计数 (Phase 0 实测偏离, 见
47 `phase0/audit-invariant-form.txt` 结论节)。 -/
48 def D4 :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 
49 | 0, mask, h => if mask = 0 ∧ h = 0 then 1 else 0
50 | n + 1, mask, h =>
51   (steps4.filter (fun t => t.2.1 = mask ∧ t.2.2 ≤ h)).map
52   (fun t => D4 n t.1 (h - t.2.2)) |>.sum
53
54 /-- **T3 主序列  $a3$  **:  $4 \times n$  的「恰半」 (累计水平砖 =  $n$ ) 走法数 = `D4
    n 0 n`。 -/
55 def a3 (n :  $\mathbb{N}$ ) :  $\mathbb{N}$  := D4 n 0 n
56
57 /-- **T3-支撑引理 (I2) **: 可达状态的掩码位数为偶 ( $pc\ mask \equiv 4 \cdot n \equiv
    0 \pmod{2}$ )。 -/
58 theorem D4_pc_even (n mask h :  $\mathbb{N}$ ) (hh : D4 n mask h ≠ 0) : 2 | pc
    mask := by
59
60 /-- **T3-支撑引理 (I1) **: 可达状态的累计水平砖数恒为偶。 -/
61 theorem D4_h_even (n mask h :  $\mathbb{N}$ ) (hh : D4 n mask h ≠ 0) : 2 | h :=
    by
62
63 /-- **T3-Z (零方向) **: ` $n` 为奇数时主序列取值为 0。 -/
64 theorem a3_odd_eq_zero (n :  $\mathbb{N}$ ) (hn : n % 2 = 1) : a3 n = 0 := by
65
66 /-- **T3-N (非零方向) **: ` $n` 为偶数时主序列取值非零 ( $4 \times 2$  块拼接构
    造)。 -/
67 theorem a3_even_ne_zero (k :  $\mathbb{N}$ ) : a3 (2 * k) ≠ 0 := by
68
69 /-- **T3-I (支撑刻画, 主定理) **: ` $a3\ n` 非零当且仅当 ` $n` 为偶数。
    -/
70 theorem a3_ne_zero_iff (n :  $\mathbb{N}$ ) : a3 n ≠ 0 ↔ n % 2 = 0 := by$$$$ 
```

This is lines 30–103 of the frozen statement snapshot with exactly five lines removed: the five proof-body placeholder lines (one per theorem). That single, stated elision is the deposit convention (see the sibling claims): the

frozen file’s bodies are the formalization department’s placeholders, and reproducing an unfinished-proof marker in this note would be rejected by the deposit’s static lint. Everything else—doc-comments including their internal notes, definitions, binders, signatures and the `:=` by keyword—is verbatim and in order. `audit/inspect-tex.py` checks this mechanically: it asserts that every remaining line occurs in the source in order, and that the only lines missing from the block are those placeholder bodies. The real, sorry-free proofs follow in excerpt.

3 Proof

The complete machine proof is 183 lines long and therefore exceeds a comfortable page; the listing below is a *contiguous excerpt*, lines 86–183 of `final/03-half4-proved.lean`, which is the entire proof development: it opens with the proof-auxiliary section, contains the five private helper lemmas (`map_sum_ne_zero_of_mem`, `exists_of_sum_ne_zero`, `steps4_guard`, `D4_inv`, `D4_step2`) that are additions on the proof side and not part of the frozen statement, and then carries all five theorems with their complete proof bodies. The material left out is lines 1–85 of the file: the header comment (1–28), the `import` line, and the re-listing of the frozen definitions (30–84), which are reproduced verbatim in the statement listing above. The full text is in the deposit under `proofs/03-half4-proved.lean`. A prose reconstruction is not an obligation of this note: narrative proofs belong to a full paper, which has been deferred.

Listing 2: Verbatim excerpt, lines 86–183 of `final/03-half4-proved.lean` (183 lines, sha256 548f3ef5...); the full text is in the deposit’s `proofs/`.

```

1  /-! ## 证明辅助（军政部 T3 添加，不改动上方冻结 statement） -/
2
3  /-- 列表 `map f` 之和非零  $\Rightarrow$  某个被映射元素的值非零。 -/
4  private lemma map_sum_ne_zero_of_mem { $\alpha$  : Type*} (f :  $\alpha \rightarrow \mathbb{N}$ ) (l
   : List  $\alpha$ ) {a :  $\alpha$ }
5      (ha : a  $\in$  l) (hfa : f a  $\neq$  0) : (l.map f).sum  $\neq$  0 := by
6      intro h0
7      exact hfa (List.sum_eq_zero_iff.mp h0 (f a) (List.mem_map.mpr ⟨a
   , ha, rfl⟩))
8
9  /-- `map f` 之和非零  $\Rightarrow$  存在一项其值非零。 -/
10 private lemma exists_of_sum_ne_zero { $\alpha$  : Type*} (f :  $\alpha \rightarrow \mathbb{N}$ ) (l :
   List  $\alpha$ )
11      (h : (l.map f).sum  $\neq$  0) :  $\exists$  a  $\in$  l, f a  $\neq$  0 := by

```

```

12 by_contra hc
13 simp only [not_exists, not_and, not_not] at hc
14 exact h (List.sum_eq_zero_iff.mpr fun x hx => by
15   obtain ⟨a, ha, rfl⟩ := List.mem_map.mp hx
16   exact hc a ha)
17
18 /-- `steps4` 的转移恒等式 (`decide` 穷举 29 条全验)：列格数偶 + 出
   掩码位数 = 伸出砖数。 -/
19 private lemma steps4_guard {occ mask j : ℕ} (ht : (occ, mask, j)
   ∈ steps4) :
20   2 ∣ (pc occ + j) ∧ pc mask = j := by
21   have hall : steps4.all
22     (fun t => decide (2 ∣ (pc t.1 + t.2.2) ∧ pc t.2.1 = t.2.2))
23     = true := by decide
24   exact of_decide_eq_true (List.all_eq_true.mp hall (occ, mask, j)
   ht)
25
26 /-- **I1 ∧ I2 (互归纳的合并不变量) **: 可达状态的累计水平砖数与掩码
   位数均为偶。 -/
27 private lemma D4_inv (n : ℕ) :
28   ∀ mask h : ℕ, D4 n mask h ≠ 0 → 2 ∣ h ∧ 2 ∣ pc mask := by
29   induction n with
30   | zero =>
31     intro mask h hh
32     by_cases hc : mask = 0 ∧ h = 0
33     · obtain ⟨rfl, rfl⟩ := hc; decide
34     · exact absurd (by simp [D4, hc]) hh
35   | succ n ih =>
36     intro mask h hh
37     have hh' : ((steps4.filter (fun t => t.2.1 = mask ∧ t.2.2 ≤ h)
38       ).map
39         (fun t => D4 n t.1 (h - t.2.2))).sum ≠ 0 := hh
40     obtain ⟨t, htmem, htne⟩ := exists_of_sum_ne_zero (fun t => D4
41       n t.1 (h - t.2.2))
42     (steps4.filter (fun t => t.2.1 = mask ∧ t.2.2 ≤ h)) hh'
43     obtain ⟨htsteps, htp⟩ := List.mem_filter.mp htmem
44     obtain ⟨hocc, hjle⟩ := of_decide_eq_true htp
45     obtain ⟨ihh, ihpc⟩ := ih t.1 (h - t.2.2) htne
46     obtain ⟨hg1, hg2⟩ := steps4_guard htsteps
47     have hdj : 2 ∣ t.2.2 := by
48       have h := Nat.dvd_sub hg1 ihpc
49       rwa [Nat.add_sub_cancel_left] at h
50     have hdh : 2 ∣ h := by
51       have h := Nat.dvd_add ihh hdj
52       rwa [Nat.sub_add_cancel hjle] at h

```

```

50     exact ⟨hdh, by rw [← hocc, hg2]; exact hdj⟩
51
52 /-- **T3-N 的构造步**：两步环  $(0, 12, 2) \rightarrow (12, 0, 0)$  把非零走法升
    2 列、加 2 砖。 -/
53 private lemma D4_step2 (n h : ℕ) (hh : D4 n 0 h ≠ 0) : D4 (n + 2)
    0 (h + 2) ≠ 0 := by
54   have h1 : D4 (n + 1) 12 (h + 2) ≠ 0 := by
55     refine map_sum_ne_zero_of_mem (f := fun t => D4 n t.1 (h + 2 -
        t.2.2))
56     (l := steps4.filter (fun t => t.2.1 = 12 ∧ t.2.2 ≤ h + 2))
        (a := (0, 12, 2)) ?_ ?_
57     · apply List.mem_filter.mpr
58       exact ⟨by decide, by simp⟩
59     · simp using hh
60   refine map_sum_ne_zero_of_mem (f := fun t => D4 (n + 1) t.1 (h +
        2 - t.2.2))
61   (l := steps4.filter (fun t => t.2.1 = 0 ∧ t.2.2 ≤ h + 2)) (a
        := (12, 0, 0)) ?_ ?_
62   · apply List.mem_filter.mpr
63     exact ⟨by decide, by simp⟩
64   · simp using h1
65
66 /-- **T3-支撑引理 (I2)**：可达状态的掩码位数为偶 ( $pc\ mask \equiv 4 \cdot n \equiv
    0 \pmod{2}$ )。 -/
67 theorem D4_pc_even (n mask h : ℕ) (hh : D4 n mask h ≠ 0) : 2 ∣ pc
    mask :=
68   (D4_inv n mask h hh).2
69
70 /-- **T3-支撑引理 (I1)**：可达状态的累计水平砖数恒为偶。 -/
71 theorem D4_h_even (n mask h : ℕ) (hh : D4 n mask h ≠ 0) : 2 ∣ h :=
72   (D4_inv n mask h hh).1
73
74 /-- **T3-Z (零方向)**：`n` 为奇数时主序列取值为 0。 -/
75 theorem a3_odd_eq_zero (n : ℕ) (hn : n % 2 = 1) : a3 n = 0 := by
76   by_contra hc
77   have h2 : 2 ∣ n := D4_h_even n 0 n (by simp [a3] using hc)
78   rw [Nat.dvd_iff_mod_eq_zero] at h2
79   omega
80
81 /-- **T3-N (非零方向)**：`n` 为偶数时主序列取值非零 ( $4 \times 2$  块拼接构
    造)。 -/
82 theorem a3_even_ne_zero (k : ℕ) : a3 (2 * k) ≠ 0 := by
83   induction k with
84   | zero => decide
85   | succ k ih =>

```

```

86     exact D4_step2 (2 * k) (2 * k) (by simp [a3] using ih)
87
88 /-- **T3-I (支撑刻画, 主定理) **: `a3 n` 非零当且仅当 `n` 为偶数。
89 -/
89 theorem a3_ne_zero_iff (n : ℕ) : a3 n ≠ 0 ↔ n % 2 = 0 := by
90   constructor
91   · intro h
92     by_contra hodd
93     have : n % 2 = 1 := by omega
94     exact h (a3_odd_eq_zero n this)
95   · intro h
96     have hn : n = 2 * (n / 2) := by omega
97     rw [hn]
98     exact a3_even_ne_zero (n / 2)

```

4 Verification evidence

- Toolchain: Lean 4 v4.34.0; mathlib4 v4.34.0, revision 5ed29652.
- Statement integrity: formalized/03-half4-statements.lean, 104 lines, sha256 b142670a02c923ff2d90e416957938e222762f139a67d5f64b593ce9064ed74b. Final artifact final/03-half4-proved.lean, 183 lines, sha256 548f3ef528b432c4c0cbfb95de7993b246d1cc782f9ec2042952af68db30036a. Every listing above is byte-identical to its source (checked by audit/inspect-tex.py, which asserts that each listed line occurs in its source in order and that the only omitted lines are the statement file's placeholder proof bodies).
- Kernel check: strict compilation exits 0 with no sorry and no admit. #print axioms output, verbatim:


```

'D4_pc_even' depends on axioms: [propext, Classical.choice, Quot.sound]
'D4_h_even' depends on axioms: [propext, Classical.choice, Quot.sound]
'a3_odd_eq_zero' depends on axioms: [propext, Classical.choice, Quot.sound]
'a3_even_ne_zero' depends on axioms: [propext, Classical.choice, Quot.sound]
'a3_ne_zero_iff' depends on axioms: [propext, Classical.choice, Quot.sound]

```

 Every line is a subset of {propext, Quot.sound, Classical.choice}; sorryAx does not occur.
- Audit chain: statement freeze (sha256), byte-wise symmetric-difference comparison against the frozen snapshot, strict-mode

compilation, and the axiom whitelist check. The comparison records 9 of 9 frozen declarations byte-identical and notes that the proof artifact adds five auxiliary lemmas, which is allowed. Originals are in the deposit under `audit/`.

5 Novelty evidence

Under the pipeline’s C9 novelty discipline the verdict for this object is *no occupying record found*. The full query log, including every zero-hit query, is in the deposit at `audit/c9-record.md`; it is reproduced verbatim from the screening batch and the present batch’s supplementary search.

Summary of channels. OEIS: the sequence, its compressed subcolumn, its even-index variant and its first difference all returned zero hits. Short prefixes of the compressed subcolumn matched two entries (A220379 and A292752), but the match breaks at the fifth term; these are short-window coincidences, do not constitute an occupying record, and are deliberately not cited here as related literature. Channel availability was established by positive controls before any zero-hit was read: A001835 and A005178 (the unconstrained $3 \times n$ and $4 \times n$ domino-tiling counts) matched. Literature: four channels were queried (arXiv API, Crossref, Mathematics Stack Exchange, zbMATH Open); there were no direct hits on this constrained-selection object. Library layer: no hits in mathlib, compfiles, or the scanned external repositories.

Two honest limitations are recorded and must not be dropped in any restatement. First, the OpenAlex channel was *unreachable* from the machine used for this search (Cloudflare-fronted hosts timed out; DNS resolution was verified correct against a public resolver, so this is not a DNS-poisoning artifact). Crossref was used as a substitute DOI-level index. Coverage is therefore *lower* than in the screening batch, and this is a channel gap, not strengthened evidence. Second, a zero hit is *not* novelty: the strongest claim licensed here is that no record of this constrained-selection object was found in the channels queried. No moment or expectation novelty is claimed: mixed moments $E[V^a H^b]$ for general $m \times n$ boards are already occupied by *Statistics of Domino Tilings on a Rectangular Board*, Fibonacci Quarterly (2019), doi:10.1080/00150517.2019.12427625 [3]. The $3 \times n$ and $4 \times n$ objects of this family are m -dimensional variants of one another and are not used as independent novelty evidence for each other; the variant partner of the present $4 \times n$ track is the $3 \times n$ half-horizontal track, which is not cited here as support.

6 Scope and limitations

The kernel-level results concern the pure finite-state integer recursion defined above. They do *not* establish that $a_3(n)$ equals the number of domino tilings of the $4 \times n$ board in which horizontal dominoes make up exactly half of all tiles. That identification is a combinatorial semantics bridge; it is stated here only as a conjecture, is not part of the theorem layer, and is supported only by numerical probes and external sequence-database evidence. The definitions themselves never mention tilings, dominoes or counting—the state is a column mask and the recursion involves only masks and integer counts—so what is formalized is a pure integer recursion, and the combinatorial bridge is still *not* in the theorem layer. Accordingly, the results are described throughout as structural theorems about a recursion, never as a proved statement about tiling counts.

The definition layer of this track was changed, under a user-approved rule-override event recorded as O-2, from a pure sequence recurrence to a finite-state integer recursion (a dynamic-programming layer) with $D_4: \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ and $a_3(n) := D_4(n, 0, n)$. This is a change of the object being formalized, not a weakening: the grading of all five theorems is *complete proof* (0 **sorry**, axiom whitelist), and no extra hypothesis was added. The absence of a constant-coefficient (C-finite) recurrence for a_3 must be read as *none found at order ≤ 60* , not as *does not exist*; the order-4 P-recursive relation reported during exploration was not brought into the frozen statement and is not claimed here, so this note claims neither a recurrence nor a closed form.

This note contains no literature survey; it is a deposit record, not a survey paper.

Data availability

Lean sources, verification and novelty-search evidence are deposited on Zenodo (DOI: **RESERVED-DOI**) and mirrored as a public snapshot repository. The note is released under CC BY 4.0 and the code under MIT.

Use of generative AI

- *AI-performed stages*: proposition generation, natural-language-to-Lean formalization, proof search, and drafting of this text.

- *Model, node, budget*: pipeline node `anthropic/a6api-main/deepseek-v4.1-flash` (wire-id; resolved at level L2 of the pipeline’s model-detection ladder, snapshot taken 2026-09-30 09:35). Source-task budget: sampling 1 of 8 allowed runs on this track (10 of 32 across four parallel tracks); judgement-class LLM calls 6 of 16 across the batch; Lean formal compilations 6 of 12 on this track (19 of 48 across the batch); LaTeX 0. The numbers are copied from the source task’s `budget.log`.
- *Final adjudication*: all statements and proofs reported here were checked by the Lean 4 kernel. Every proof compiles with no `sorry`, and `#print axioms` reports only `{propext, Quot.sound, Classical.choice}`. Statement text was frozen before proving and compared byte-wise afterwards.
- The author reviewed the material and is responsible for all content. AI systems are not listed as authors.

Author contributions

The author determined the research question and the scope fences, verified the semantics of the statements, and is responsible for signing off and for any deposit or release. The automated pipeline (LLM-driven) generated the proposition, the formalization, the proof search and the draft text. No external release is performed by the pipeline.

References

- [1] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. *CADE-28*, 2021. https://doi.org/10.1007/978-3-030-79876-5_37
- [2] The mathlib Community. The Lean mathematical library. *CPP 2020*, 2020. <https://doi.org/10.1145/3372885.3373824>
- [3] *Statistics of Domino Tilings on a Rectangular Board*. Fibonacci Quarterly, 2019. <https://doi.org/10.1080/00150517.2019.12427625>